Security Hardening

What the Pi Zero 2 W build and the common tools do to keep a node safe: how people log in, how a request is checked before it runs, which headers each page sends, which requests are refused, and where the protection stops.

LOGIN | REQUEST CHECKS | HEADERS | GAPS

SUMMARY · READ-ONLY REVIEW

Part 1 of 3. Sources: the pi02w tools (nodecontrol, sysmon, wifimon, 44helper), the common tools (instmon, pibrary, Node Talker, Node Recorder) on branch ccr-9d4fe19f-ssud83. Nothing was run on a node.

How a person gets past the login

1.1 The root password

The Dashboard (nodecontrol) checks the root password first through the system's PAM stack, using the `login` service. If PAM is not usable, it falls back to reading the root hash from `/etc/shadow` and checking it with `crypt(3)` from libcrypt. The logging of the password itself was not traced in this review.

The other web tools use the same two-stage check against the root account: Node Talker, Node Recorder, instmon, pibrary, SysMon and 44helper each try the PAM `login` service through the Python `pam` module, with the shadow-hash fallback. wifimon does not use that module. It talks to libpam directly through `ctypes`, with its own PAM service name (`wifimon`), and falls back to the shadow hash (compared in constant time) when PAM gives no answer. A hash that starts with `!` or `*` (a locked password) never matches. 44helper's login needs the root password as well.

1.2 Lockouts

A failed login counts against the address it came from. Two rules are in use:

TOOLS	RULE
Dashboard, SysMon, 44helper	Five failures within five minutes lock the address out for 15 minutes (`_LOGIN_LOCKOUT_SEC = 15 * 60`)
instmon, wifimon, pibrary, Node Talker, Node Recorder	The first four failures carry no wait. From the fifth, the wait is 5 seconds and doubles with each further failure, capped at 300 seconds. instmon answers a locked request with 429 and a `Retry-After` header

The lockout is keyed on the client address the server sees (`client_address`). A proxy in front of the node would put every user behind one address, so the proxy rules in Part 5 matter here too.

The Dashboard and SysMon share one record of sessions, failures and lockouts in `/run/asl_dvs/auth_session.json` (folder mode 0700, file mode 0600, locked with `flock` while it is changed). A lockout on one is therefore a lockout on the other. The other tools keep their own failure counts in memory.

Node Talker and Node Recorder also cap logins from outside the local network as a group: after 20 failed internet logins in 10 minutes, every internet login waits until that window ends, whoever the client is.

1.3 Connection caps

Three servers refuse new connections when too many are open. The check runs before a request is read, and a refused connection is simply dropped.

TOOL	OPEN CONNECTIONS	FROM ONE ADDRESS
Dashboard	48	16
SysMon	24	12
wifimon	16	8

The Dashboard logs each refusal, and SysMon logs at most one refusal a minute. A search of 44helper, instmon, pibrary, Node Talker and Node Recorder found no such cap.

1.4 Session cookies

After a good login the server sets a session cookie. Each cookie is sent with `HttpOnly`, `SameSite=Strict`, `Path=/` and a `Max-Age`. A script on the page cannot read it, and a browser does not send it on a request that starts on another site. wifimon adds `Secure` when it runs with TLS, and its TLS setting is off by default.

TOOL	COOKIE NAME	SESSION LIFETIME
Dashboard, SysMon	`asl_dvs_session` (shared)	12 hours
44helper	`helper44_session`	2 hours, with the cookie refreshed on use (cookie life 1 year)
instmon	`instmon_session`	12 hours
wifimon	`wifimon_session`	12 hours
pibrary	`pibrary_session`	12 hours
Node Talker	`nodetalker_session`	12 hours
Node Recorder	`noderecorder_session`	12 hours

Cookies are not tied to a port, so a browser sends the shared Dashboard and SysMon cookie to both. That is how one login serves both tools.

Refusing requests that look forged

A web page that a person has open can be made to send a request on their behalf. The tools block that in two ways: they demand a header that a plain web form cannot send, or they demand a content type that a plain form cannot send.

CHECK	WHERE	REFUSED WITH
`Content-Type: application/json` **required**	Dashboard, SysMon	415
`X-Requested-With: XMLHttpRequest` **required**	44helper, instmon, pibrary, Node Talker, Node Recorder, amp Node Talker	403 (Missing or invalid X-Requested-With header)
Header, JSON content type and a matching `Origin`	wifimon (`_csrf_ok`)	Refused. If an `Origin` header is present, its host must equal the `Host` header

2.1 Why these work

A browser form can send a plain POST with a few content types, and it cannot add a custom header to a request from another site without a permission check that fails. Requiring either one closes the common route.

The check is only as good as its coverage. The Dashboard and SysMon use the content-type check, and the other tools use the header check. A new endpoint added without the check has no protection, so each new route needs the same guard.

The local key and the loopback-only API

3.1 Loopback peers only

The Dashboard's local API accepts a request only from a loopback address (`127.0.0.1`, `::1`, or the IPv4-mapped form). Any other peer is refused, even if it has the key.

3.2 The key

The local API also needs the header `X-ASL-DVS-Local-Key`. The key is kept in `/etc/asl_dvs/local_api.key`. A key of at least 32 characters is used as it is. If the file is missing, the Dashboard creates a 64-character hexadecimal key with mode 0600. If the file exists but holds fewer than 32 characters, the code does not replace it and returns what it holds, so the file must be kept in good order.

The name of the header and the path keep the `asl_dvs` system name, as the naming rules require.

What each page tells the browser

HEADER	PURPOSE	SET BY (FOUND IN SOURCE)
`X-Frame-Options`	Stops the page being shown inside another site's frame (clickjacking)	`DENY` in the Dashboard, 44helper, wifimon, instmon and pibrary; `SAMEORIGIN` in Node Talker and Node Recorder
`X-Content-Type-Options: nosniff`	Stops the browser guessing a file's type	All eight: Dashboard, SysMon, 44helper, wifimon, instmon, pibrary, Node Talker, Node Recorder
`Referrer-Policy: no-referrer`	The page does not send the address it came from	44helper, wifimon, instmon, pibrary
`Content-Security-Policy`	Limits which scripts, styles and connections the page may use	Dashboard, 44helper, wifimon, instmon, pibrary

The Dashboard's policy is one fixed text sent with its page: `default-src 'self'`, inline scripts and styles allowed, connections to itself only, and no framing (`frame-ancestors 'none'`). SysMon sets only `nosniff`, and Node Talker and Node Recorder set only `nosniff` and the frame header. Those three send no Content-Security-Policy or Referrer-Policy header (source search; not confirmed on a running node).

Requests that came through a proxy or tunnel

A request that passes through a reverse proxy or a tunnel such as Cloudflare carries a header that names the proxy. Several tools refuse those requests on purpose, so a remote visitor cannot reach a control through a tunnel that puts them on the local network. wifimon and the 44helper terminal refuse any request that carries one of their listed headers, from any address. Node Talker and Node Recorder treat a request as proxied only when it comes from a loopback address (a tunnel program on the node itself) *and* carries one of their headers.

TOOL	HEADERS CHECKED	EFFECT
wifimon	X-Forwarded-For, X-Forwarded-Host, X-Forwarded-Proto, X-Real-Ip, Forwarded, Via, Cf-Connecting-Ip, Cf-Ray, Cf-Visitor, Cf-Ipcountry	Control actions refused with 403
Node Recorder	Forwarded, X-Forwarded-For, X-Real-IP, CF-Connecting-IP, Tailscale-Funnel-Request	A loopback request with one of them is treated as coming from outside
44helper terminal	Cf-Connecting-Ip, Cf-Ray, Cf-Ipcountry, Cf-Visitor, X-Forwarded-For and others	Terminal refused with a message that it works only from the local network
Node Talker	The same five as Node Recorder (Forwarded, X-Forwarded-For, X-Real-IP, CF-Connecting-IP, Tailscale-Funnel-Request)	A loopback request with one of them is treated as coming from outside. A request from a LAN address with one of them is still treated as local

The lists are not the same in every tool. A header missing from one list is not checked there. The lists are best read as a tripwire, and the book should not present them as a complete defence against a tunnel.

Plain HTTP, root services and the terminal

6.1 Plain HTTP for most tools

TOOL	TRANSPORT
Node Talker, Node Recorder	HTTPS by default (`https: True`), with a self-signed certificate made on first start. If the certificate cannot be loaded, they fall back to plain HTTP and the page warns that the password is sent in plain text. A plain HTTP request to the same port is redirected to HTTPS.
wifimon	Has TLS code, switched off (`USE_TLS = False`). It binds to `0.0.0.0`.
Dashboard, SysMon, instmon, 44helper, pibrary	No TLS code in the file: plain HTTP only. The Dashboard binds to `0.0.0.0`. instmon's install says it serves plaintext HTTP only and should sit on a trusted LAN or VPN, or behind a TLS-terminating proxy.

Plain HTTP means a login password and a session cookie cross the local network unencrypted. On a home LAN that is a known risk, and the book should state it rather than imply the logins are encrypted. The session cookie gets `Secure` only when a page is served over TLS (wifimon) or, for Node Talker, when the request came over TLS or through a proxy that says https.

6.2 Services run as root

The 44helper unit file sets `User=root`. The other tools' unit text sets no `User=` line, so systemd runs them as root too. A search of all eight tool files found no sandboxing directive (`NoNewPrivileges`, `ProtectSystem`, `ProtectHome`, `PrivateTmp`, `CapabilityBoundingSet` or `ReadWritePaths`). A flaw in any of these web services therefore has root on the node. The units do set `Restart=always` and a watchdog, which keep a service up but do not limit what it can do.

6.3 The terminal is a root shell

The 44helper terminal opens a shell in the browser. Before it opens, the server checks that the request is not proxied, that it comes from loopback, link-local or a private address, that the service runs as root, and that the terminal-enable switch is set. Each open also needs the root password again (a wrong one counts against the login lockout) and a one-use ticket that lasts 60 seconds, and the WebSocket must have an Origin that matches its Host. A session closes after 10 minutes without typing. The terminal is the most sensitive control in the pi02w build, and the book should describe it as root access gated by the network and the switch.

The 44helper job worker runs commands through a shell (`shell=True`). The install and purge modes of a tab action are held to the page's own steps: the command text must equal the step's command, or the job is refused with a "step changed" message. The **custom** mode has no such list. It runs the command text the page sends, as root, after a root check, a masked-service check for package changes, and a package-health check. A logged-in session can therefore run any command through that mode, so the book should call 44helper's login a root login.

What the full build has that the pi02w build does not

The full build of SysMon (`sysmon_v6_13_95_20261028.py`) includes a security audit of 14 checks: USRP, M17 and AMI port exposure, AMI default credentials, IAX2 secret strength, the BrandMeister hotspot password, chan_sip disabled, SSH hardening, Fail2ban jails, Monit authentication, Dashboard web authentication, Asterisk file permissions, service not running as root, and unit file permissions. The pi02w build of SysMon (`sysmon_pi02w_v6_13_83_20261028.py`) has no audit table.

This summary changes no code. The gap is recorded here so the book does not describe the audit as a feature of both builds. Porting the audit to the pi02w build is a separate change, and under the repo rules it needs a request that names the pi02w build.

What the hardening does and does not cover

8.1 Layers, not one switch

Login, lockout, the request checks, the local key, the header set and the proxy refusals each stop a different route. None of them is a complete defence alone, and each tool sets a different subset.

8.2 The LAN is the outer wall

Most controls assume the node is on a trusted home or office LAN. Any device on that LAN can reach the plain HTTP ports. The LAN-only policy (the second summary in this series) describes that wall.

8.3 Not checked

- The PAM configuration of the `login` service on a running node.
- Every endpoint in every tool for the request checks in Part 2. Only the check functions were read.
- Whether a browser on a real node receives the headers listed in Part 4.
- How 44helper stores its sessions (it keeps its own failure counts in memory).